Studio Nx

백엔드 309 — 템포 서비스

2026-10-21

백엔드 309 — 템포 서비스 (`services/tempo_service.py`)

항목	내용
파일	`studionx/backend/app/services/tempo_service.py`
작업	S1-13 템포 변경: 템포 변경점 읽기와 통째 교체, 권한 확인 (PRD 02 FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 308 모델과 스키마, 다음 310 API와 `/full`

1. 하는 일

함수	하는 일	권한
`tempo_items`	프로젝트의 템포 변경점을 tick 오름차순으로 읽습니다. 권한 확인은 부르는 쪽이 합니다.	없음
`list_tempo_changes`	멤버인지 확인한 뒤 목록을 돌려줍니다.	멤버면 누구나 (VIEWER 포함)
`replace_tempo_changes`	지금 변경점을 모두 지우고 보낸 목록으로 바꿉니다.	OWNER, EDITOR만

```
replace_tempo_changes(project_id, 보낸 목록)
  멤버 확인 (아니면 404) —▶ OWNER 또는 EDITOR 확인 (아니면 403)
  한 트랜잭션 안에서: 이 프로젝트의 변경점 전부 삭제 —▶ 보낸 목록 삽입 —▶ 커밋
  DB에서 새로 읽어 tick 오름차순으로 돌려줌 (빈 목록을 보내면 모두 지워 평평한 템포로 돌아감)
```

2. 언제 쓰나

쓰는 곳	하는 일
`GET /projects/{id}/tempo-changes` (310)	`list_tempo_changes`
`PUT /projects/{id}/tempo-changes` (310)	`replace_tempo_changes`
`GET /projects/{id}/full` (310)	권한을 이미 확인했으므로 `tempo_items`로 변경점만 읽습니다.

3. 코드

```
def _item(row: TempoChange) -> TempoChangeItem:
    """TempoChange → TempoChangeItem. bpm 은 DECIMAL 을 숫자(float)로 바꿔 보낸다."""
    return TempoChangeItem(
        tempo_change_id=row.tempo_change_id,
        tick=row.tick,
        bpm=float(row.bpm),
        ramp=row.ramp.value,
    )
```

```
def tempo_items(db: Session, project_id: int) -> list[TempoChangeItem]:
    """프로젝트의 템포 변경점을 tick 오름차순으로 읽는다 (권한 확인은 부르는 쪽이 한다).
```

```
    (구현: AI — 2026-10-21, S1-13) 통째 교체 직후에는 세션에 남은 옛 목록과 섞이지 않게
    관계 대신 쿼리로 DB 에서 새로 읽는다.
    """
    rows = db.scalars(
        select(TempoChange).where(TempoChange.project_id == project_id).order_by(TempoChange.tick)
    ).all()
    return [_item(row) for row in rows]
```

```
def list_tempo_changes(db: Session, user: User, project_id: int) -> TempoChangesResponse:
    """프로젝트의 템포 변경점 목록 (FR-202a). 멤버면 누구나."""
    project_service.get_membership(db, project_id, user)
    items = tempo_items(db, project_id)
    return TempoChangesResponse(items=items, total=len(items))
```

```
def replace_tempo_changes(
    db: Session, user: User, project_id: int, data: TempoChangesSaveRequest
) -> TempoChangesResponse:
    """템포 변경점을 보낸 목록으로 통째 교체한다 (FR-202a). OWNER·EDITOR 만.

    (구현: AI — 2026-10-21, S1-13) 지금 붙은 변경점을 모두 지우고 보낸 것을 새로 넣는다.
    한 트랜잭션이라 반쪽 상태가 남지 않는다. 빈 목록이면 모두 지워 처음의 평평한 템포로 돌아간다.
    같은 위치(tick) 중복은 요청 검증(422)이 먼저 막고, 삭제와 삽입을 같은 트랜잭션에서 하므로
    UNIQUE(project_id, tick) 에 걸리지 않는다.
    """
    project, member = project_service.get_membership(db, project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)

    db.execute(delete(TempoChange).where(TempoChange.project_id == project.project_id))
    db.add_all(
        TempoChange(
            project_id=project.project_id,
            tick=change.tick,
            bpm=Decimal(str(change.bpm)),
            ramp=TempoRamp(change.ramp),
        )
        for change in data.changes
    )
    db.commit()
    items = tempo_items(db, project.project_id)
    return TempoChangesResponse(items=items, total=len(items))
```

4. 왜 이렇게

선택	이유
점 하나씩 고치지 않고 목록을 통째로 교체합니다	템포 줄(Tempo lane)은 점을 끌고, 더하고, 지우는 편집이라 곡선 전체가 하나의 상태입니다. 통째로 보내면 서버가 "무엇이 바뀌었는지"를 계산할 필요가 없고, 요청이 중간에 실패해도 반쪽짜리 곡선이 남지 않습니다. 노트를 통째로 바꾸는 `PUT /patterns/{id}/notes`와 같은 방식입니다.
삭제와 삽입을 한 트랜잭션에서 합니다	같은 `tick`에 새 점을 넣을 때 `UNIQUE(project_id, tick)`에 걸리지 않습니다. 같은 요청 안의 중복 `tick`은 스키마가 먼저 422로 막습니다.
교체한 뒤 DB에서 새로 읽어 돌려줍니다	세션에 남은 옛 객체와 새로 넣은 행이 섞이지 않게 하려는 것입니다 (`replace_notes`와 같은 이유). 응답이 곧 저장된 값이라 화면이 그대로 믿고 쓸 수 있습니다.

선택	이유
읽기 함수를 둘로 나눴습니다	`GET /full`은 이미 멤버 확인을 마쳤으므로, 권한 확인이 들어 있는 함수를 다시 부르면 같은 조회가 두 번 일어납니다. 권한 없는 `tempo_items`와 권한 있는 `list_tempo_changes`로 나눠서 쿼리를 한 번만 씁니다.
남의 프로젝트는 404입니다	있는지조차 알려 주지 않는 기존 규칙(`get_membership`)을 그대로 따릅니다. VIEWER는 읽을 수 있지만 저장하면 403입니다.
`bpm`은 `Decimal(str(값))`로 저장합니다	`float`을 그대로 `Decimal`로 바꾸면 0.1 같은 값에 긴 꼬리가 붙습니다. 문자열을 거치면 스키마가 반올림한 값 그대로 들어갑니다.

5. 시험

`studionx/backend/tests/test_tempo_routes.py`가 서비스를 라우터를 통해 확인합니다.

- 비어 있으면 `{items: [], total: 0}`, 저장하면 tick 오름차순으로 돌아옵니다. ramp를 생략하면 `STEP`입니다.
- 교체하면 옛 변경점이 사라지고, 빈 목록을 보내면 모두 지워집니다.
- VIEWER는 읽을 수 있고 저장하면 403이며 값은 그대로입니다. EDITOR는 저장할 수 있습니다.
- 멤버가 아니거나 없는 프로젝트는 읽기와 저장 모두 404이고, 로그인하지 않으면 401입니다.